

# Module Documentation Report

CSE396 – Computer Engineering Project

Group 7

Project: Autonomous First Responder Fire & Rescue Robot

Module 2 – AI & Vision Pipeline

## 2. Module 2 – AI & Vision Pipeline

### 2.1. Module Description

Module 2 is responsible for the visual perception and victim-analysis layer of the robot. It runs on the Raspberry Pi 5 and processes live RGB frames captured by the Pi Camera Module V3. The main purpose of this module is to detect human presence in real time and classify the detected person as **TRAPPED**, **LYING**, **STANDING**, or **NONE**. Since the project targets disaster environments where cloud connectivity may be unavailable, the entire pipeline is designed to work on-device as an edge-AI component.

Within the overall system, Module 2 acts as the robot's visual intelligence layer. It transforms raw image frames into structured victim information that can be used by the higher-level FSM, the Web Dashboard, and the Unity Digital Twin.

### 2.2. Main Responsibilities

The main responsibilities of Module 2 are:

- initializing and managing the Pi Camera Module V3,
- capturing live frames from the environment,
- detecting human candidates using a YOLO-based detector,
- selecting the most relevant target when multiple people appear,
- classifying victim condition using an edge-side analysis backend,
- producing structured output for upper modules,
- supporting dynamic FPS switching for resource-aware execution,
- pausing and resuming the vision pipeline during STT execution.

### 2.3. Hardware and Software Stack

This module is deployed on a **Raspberry Pi 5 (8GB)** with an **Active Cooler** and uses the **Pi Camera Module V3** as its main visual input source.

The software stack is Python-based and includes:

- Python 3.10+
- OpenCV
- Ultralytics YOLOv8

- Moondream2 or an alternative lightweight edge-side classifier backend

This design supports the project goal of running perception directly on the robot under edge hardware constraints.

## 2.4. Public Interface and Data Contract

The public interface of the module is exposed through the vision pipeline abstraction. External modules are expected to communicate only with this public layer instead of directly calling internal helper files.

| Method                                | Purpose                                                                          |
|---------------------------------------|----------------------------------------------------------------------------------|
| <code>initialize_camera()</code>      | Initializes the camera and loads the required inference backends.                |
| <code>get_latest_target()</code>      | Returns the latest victim result, or <code>None</code> if no person is detected. |
| <code>pause_vision_pipeline()</code>  | Temporarily stops frame processing and inference during STT execution.           |
| <code>resume_vision_pipeline()</code> | Restarts the vision pipeline after the interruption ends.                        |

The main public output type is **TargetData**. It contains:

- `pos_x, pos_y`: pixel coordinates of the target,
- `distance_cm`: estimated distance to the target,
- `severity`: victim state,
- `confidence`: model confidence score.

This output is intentionally compact so it can be consumed easily by upper layers. It is also designed to remain compatible with the victim-status and visualization needs of the higher-level modules. The public API is intentionally kept minimal, while lower-level camera, detection, and analysis operations are handled internally by specialized helper modules.

## 2.5. Internal Architecture

The public orchestration point of the module is `ai_vision.py`. Internally, the module is divided into smaller files to separate responsibilities and improve maintainability:

- `camera_internal.py`: camera initialization, frame capture, FPS update, and cleanup,
- `human_detector_internal.py`: YOLO-based person detection and best-target selection,
- `victim_analyzer_internal.py`: victim-state classification and priority-compatible output generation.

In addition, `ai_vision.py` also contains runtime control logic such as exploration/assessment switching, dynamic FPS control, and pause/resume handling during STT interrupts.



Figure 1: Internal processing flow of Module 2

## 2.6. Operational Flow

At runtime, the module first initializes the camera and required AI backends. It then continuously captures RGB frames from the environment. Each frame is processed by the human detector. If no person is found, the cycle ends without producing a target. If one or more people are detected, the most relevant candidate is selected and forwarded to the victim-analysis stage. The analysis result is then packaged into a `TargetData` object and delivered to upper modules and the FSM.

This makes Module 2 a perception pipeline rather than a simple camera reader. Its output directly supports target prioritization, visualization, and higher-level mission decisions.

## 2.7. Inter-Module Connections

Module 2 has several important connections within the project architecture:

- **Module 1 – Embedded & Hardware:** Module 2 relies on the hardware platform prepared by Module 1, including stable Raspberry Pi deployment, camera integration, power context, and robot-level embedded infrastructure. Module 2 does not directly handle low-level actuator or reflex control, but it depends on this hardware layer to operate reliably.
- **Module 3 – Acoustics & Navigation:** Module 2 works together with Module 3 as part of the tri-modal sensor-fusion strategy. Visual detections produced by Module 2 become one of the triggers of target prioritization, while Module 3 contributes acoustic localization and navigation behavior for approaching the detected target.
- **Module 4 – Web Dashboard & STT:** This is the strongest software-level interaction of Module 2. During STT execution, Module 2 may pause the vision pipeline to release CPU and RAM resources, then resume safely afterward. This interaction is necessary for resource-aware execution on edge hardware.
- **Module 5 – Unity Digital Twin:** Module 5 consumes victim-status outputs from Module 2 to display color-coded target information and operator-facing visualization. Therefore, the severity output of Module 2 must remain compatible with the priority and color mapping used by the Unity side.
- **Higher-Level FSM / Control Logic:** The FSM uses the structured outputs of Module 2 for victim scene analysis, evaluation, and action planning. Module 2 provides perception results, while the FSM is responsible for higher-level mission decisions.

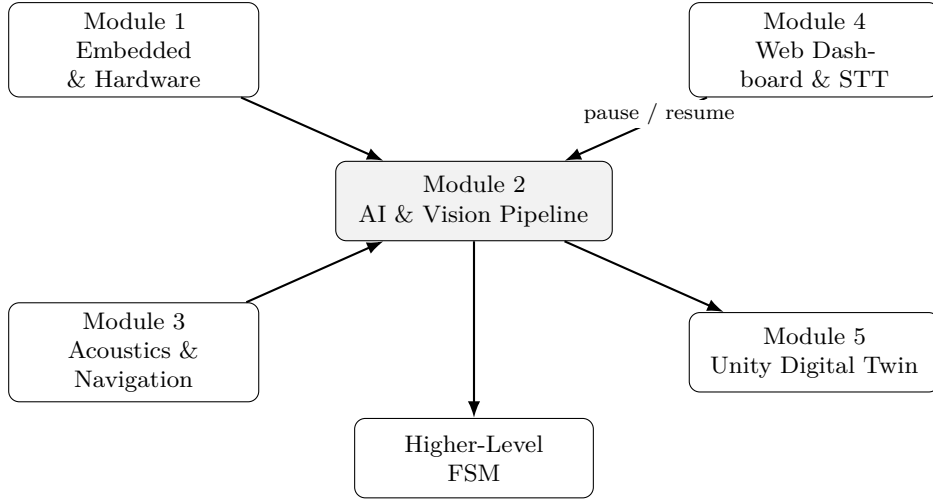


Figure 2: Main connections of Module 2 with other modules

## 2.8. Input / Output Summary

**Inputs:** live RGB frames from the camera, runtime control commands such as pause/resume, and configuration parameters such as FPS or confidence thresholds.

**Outputs:** `TargetData` objects, victim severity labels, confidence values, estimated target position and distance, and integration-ready outputs for the dashboard, Unity, and FSM.

These outputs are consumed by upper layers for visualization, reporting, and higher-level control. They also support augmented status reporting and priority-based visualization in upper layers.

## 2.9. Testing and Validation

The following test scenarios are suitable for Module 2:

- **AV-01:** Pi Camera initializes successfully on Raspberry Pi 5.
- **AV-02:** Frames are captured continuously without freeze or crash.
- **AV-03:** YOLO detector identifies a human in a test frame.
- **AV-04:** The pipeline returns `None` when no human is present.
- **AV-05:** Victim analysis distinguishes `TRAPPED`, `LYING`, and `STANDING`.
- **AV-06:** Target selection behaves consistently in multi-person scenes.
- **AV-07:** Dynamic FPS switching works correctly.
- **AV-08:** Pause/resume works safely during STT execution.
- **AV-09:** Output format remains compatible with upper modules.

## 2.10. Risks, Limitations, and Conclusion

One important risk is thermal and memory pressure on the Raspberry Pi 5, especially during heavy victim-analysis inference. Another risk is latency if the selected model is too large for real-time execution. To reduce these problems, the module uses dynamic FPS management, active cooling, and pause/resume behavior during STT.

At the current stage, final benchmark comparison between alternative victim-analysis backends is still pending. In addition, more advanced multi-target orchestration may later require deeper FSM support.

Overall, Module 2 is a central perception component of the project. It converts raw visual data into structured victim information and provides a reliable bridge between sensing and decision-making in the overall rescue workflow.